

AUTOMATED

Multi-Agentic Autonomous Trading System

Technical Documentation Manual — Publication-Grade Edition

BSCS (R) Final Year Project • 2022-26

Field	Detail
Project Title	Automated Multi-Agentic Autonomous Trading System
Author 1	Abdul Qadeer Khan — COMPF22BSR47
Author 2	Ahmad Abdullah — COMPF22BSR43
Author 3	Faisal Rehman — COMPF22BSR29
Programme	Bachelor of Science in Computer Science (BSCS) (R)
Submission	June 2026
Repository	github.com/Ahmadthealdo/Automated-Multi-Agentic-Trading-System
Deployment	Render.com Free-Tier • Neon PostgreSQL Serverless
Tech Stack	Python FastAPI • OpenAI Agents SDK • SQLAlchemy • yfinance • Langfuse • Tailwind CSS
License	Apache 2.0

Executive Summary

Automated is a production-deployed, event-driven, multi-agent AI system engineered to perform autonomous financial market analysis and generate structured, compliance-audited trading decisions in real time. The platform is built on Python FastAPI as the web application gateway, SQLAlchemy ORM over a Neon PostgreSQL serverless database, the OpenAI Agents SDK for agent orchestration, Langfuse for distributed telemetry, yfinance for deterministic OHLCV data extraction, and a Bloomberg-inspired Tailwind CSS/JS frontend featuring an interactive SVG candlestick charting engine. The system is deployed live on Render's Free-tier cloud infrastructure.

At its architectural core, Automated embodies the paradigm shift from monolithic single-prompt LLM pipelines where one context window must simultaneously handle market analysis, risk compliance, and decision synthesis toward a Decoupled Multi-Agent Asynchronous Orchestration model. Each intelligent node operates in an isolated execution sandbox with a precisely scoped system prompt and a strictly defined Pydantic output schema. This separation of concerns eliminates inter-domain context contamination, preserves token budget efficiency, and enables institutional-grade auditability via Langfuse nested execution spans.

- Context Contamination Prevention: Each agent context window contains only domain-relevant information, eliminating cross-domain token pollution that degrades reasoning fidelity in monolithic pipelines.

- Prompt Token Budget Preservation: Decoupled agents maintain O(1) constant context complexity per node, avoiding the exponential growth characteristic of monolithic approaches.
- Full Auditability: Every agent invocation is independently traceable via Langfuse nested spans — a non-negotiable requirement for financial systems subject to compliance scrutiny.
- Strict Compliance Enforcement: The Risk Manager's APPROVED/REJECTED gate is enforced deterministically at the Router layer, preventing any operator from bypassing capital preservation rules at runtime.

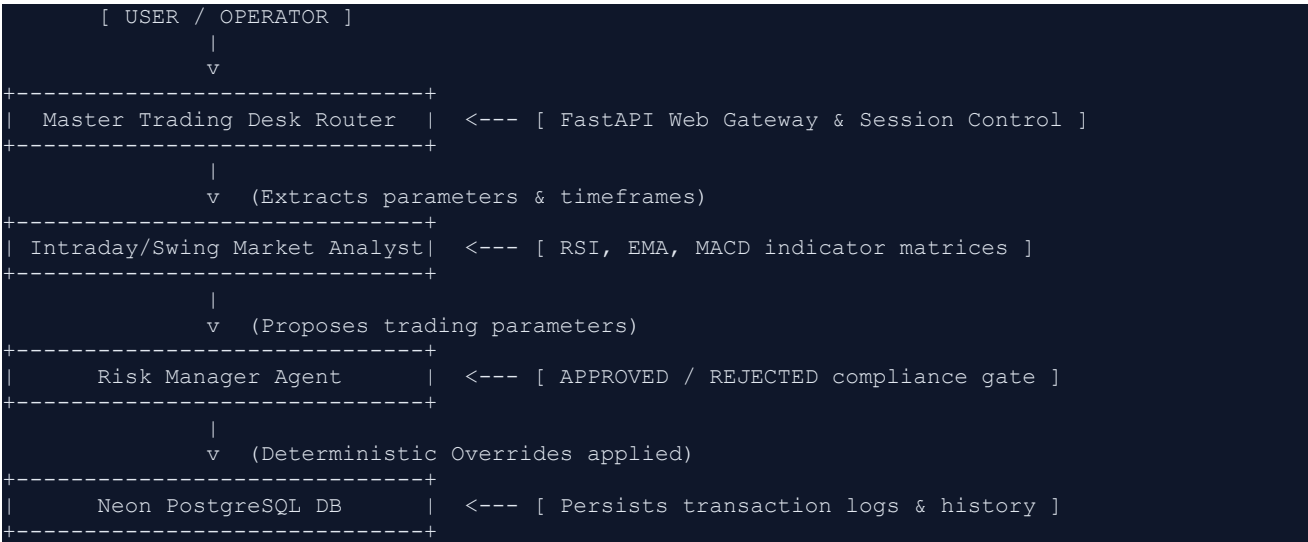
The system implements three active agent nodes: the Master Trading Desk Router (Orchestrator), the Intraday and Swing Market Analyst (Signal Synthesizer), and the Risk Manager Agent (Compliance Auditor). All inter-agent communication is mediated exclusively through typed Pydantic schemas, eliminating ambiguous natural-language handoffs between nodes.

System Architecture & Decoupled Orchestration Flow

2.1 Architecture Philosophy

Rather than relying on a single monolithic prompt, Automated enforces a strict separation of concerns among specialised execution nodes operating in sandboxed contexts. Data flows strictly via immutable Pydantic message states — no agent has visibility into another agent's reasoning context, prompt, or intermediate outputs. This architectural decision prevents a well-documented failure mode in monolithic LLM pipelines where injected risk parameters bias signal synthesis toward compliance-friendly rather than market-truthful outputs.

2.2 High-Level System Flow (from README)



2.3 N8N Agent Workflow — Live Canvas

The screenshot below shows the actual deployed N8N multi-agent workflow canvas. The Trading Desk Manager (Orchestrator) sits at the top, connected via dashed lines to the Market Analyst Tool and Risk Manager Tool sub-agents. Each sub-agent is equipped with its own Chat Model (Google Gemini), Simple Memory node, and Code Tool — mirroring the decoupled sandboxing described in the architecture.

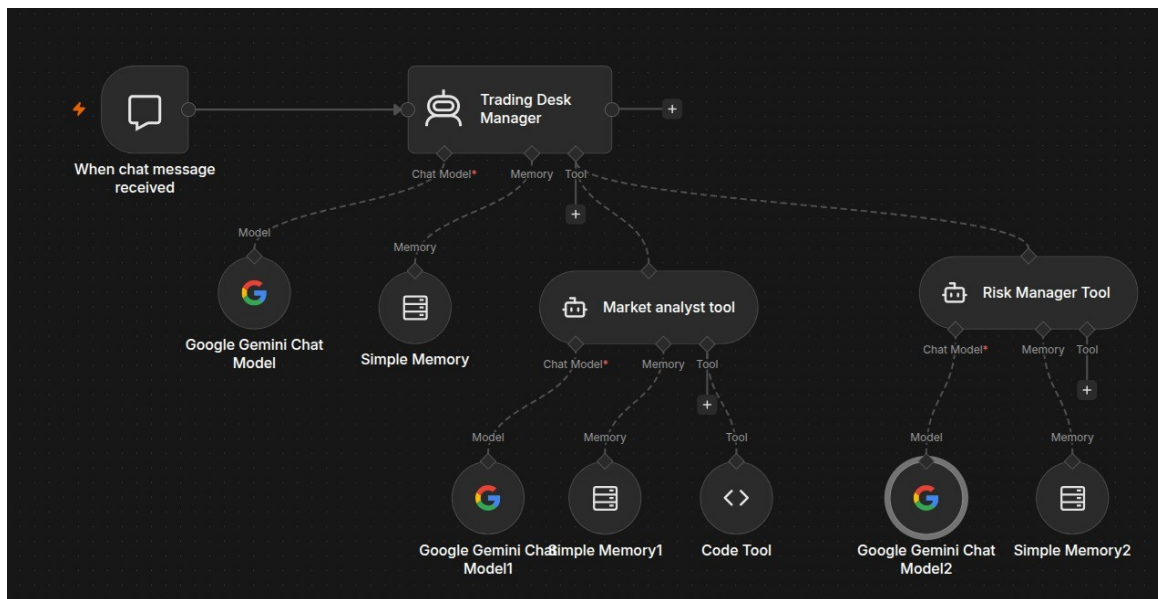


Figure 1: N8N Multi-Agent Workflow Canvas — Trading Desk Manager orchestrating Market Analyst & Risk Manager nodes with isolated memory and tool access

2.4 Multi-Agent Decoupled Orchestration Flow

The diagram below maps the precise information lifecycle: each arrow represents a Pydantic schema handoff between sandboxed agents, not shared context. The Risk Manager compliance gate enforces APPROVED or REJECTED status, with a deterministic Router override forcing HOLD at 0% allocation on rejection.

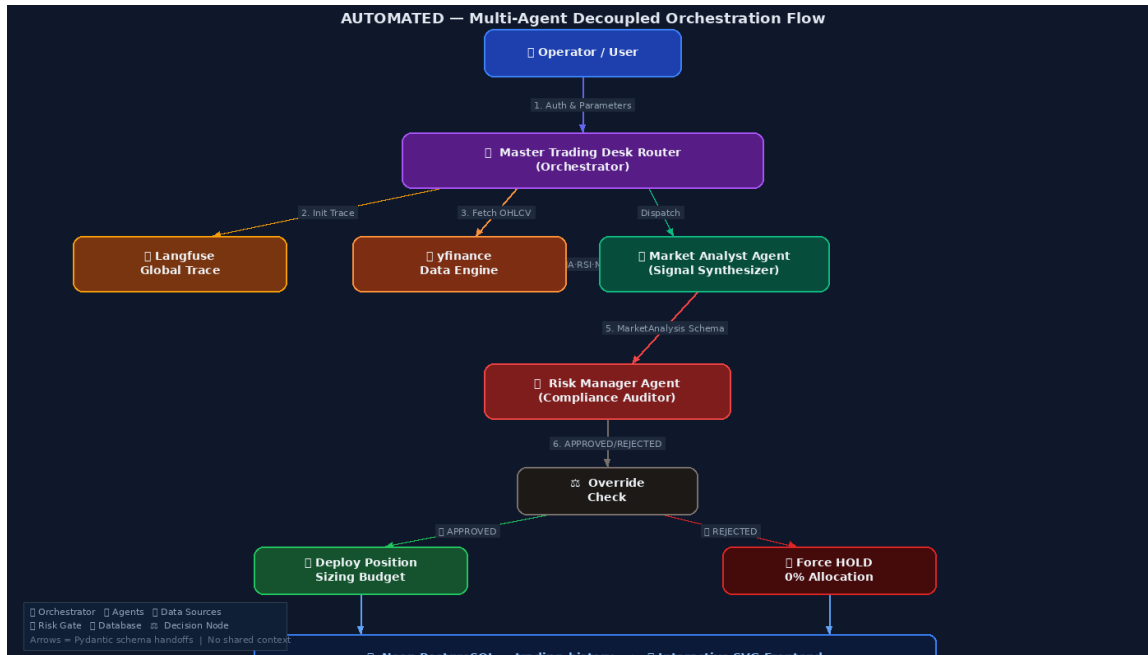


Figure 2: Multi-Agent Decoupled Orchestration Flow — Pydantic schema handoffs, compliance gate, and persistence pipeline

2.5 Pipeline Stage Breakdown

Stage 1 — Authentication & Parameter Ingestion

The operator authenticates via bcrypt-validated credentials against Neon PostgreSQL. Upon success the frontend session is unlocked. The operator submits: ticker symbol (e.g. AAPL, BTC-USD), strategy class, and capital allocation. The Router standardises tickers and validates parameters before dispatch.

Stage 2 — Langfuse Trace Initialisation

The Router immediately instantiates a Langfuse global execution trace, registering operator identity, input parameters, and a monotonic UTC timestamp. All subsequent agent calls are nested as child spans beneath this parent trace, providing complete hierarchical audit coverage.

Stage 3 — yfinance OHLCV Extraction & Indicator Computation

The Router resolves the lookback period and candle interval from the Dynamic Strategy Matrix. It invokes yfinance to fetch OHLCV data and computes EMA crossovers, MACD histograms, and RSI values. The output is serialised into a structured dictionary injected directly into the Market Analyst's user prompt.

Stage 4–5 — Market Analyst (Zero Execution Authority)

The Market Analyst holds ZERO portfolio execution authority. It cannot invoke tools, write to databases, or trigger downstream actions. Its sole mandate is signal synthesis. It returns a MarketAnalysis Pydantic object with directional bias, entry price, TP/SL levels, and a confidence score between 0.0 and 1.0.

Stage 6–8 — Risk Gate, Persistence & SVG Render

The Risk Manager applies institutional compliance rules to produce a RiskEvaluation schema. If REJECTED, the Router forces HOLD with 0% allocation before any database write. The FinalTradingDecision is then persisted to Neon PostgreSQL and the SVG chart is re-rendered with TP/SL overlays via the 10-second polling interval.

Agent Registry & Persona Specifications

3.1 Governance Framework

Each agent in Automated is governed by immutable Operational Laws encoded directly into its system prompt. These laws define the agent's epistemic boundary, output authority, communication protocol, and ethical constraints. Violating an Operational Law produces a Pydantic ValidationError, which the Router catches and converts to a safe HOLD decision rather than propagating a malformed signal to the database.

3.2 Node 1 — Master Trading Desk Router (Orchestrator)

Role & Mandate

Role: Conversational Gateway, State Initiator & Interface Controller

Objective: Ingests raw user intent, validates asset token parameters, and guides execution preferences

Rules: Standardises tickers (AAPL for equities, BTC-USD for crypto); restricts data fetching until asset identity and timeframe are locked in

- Operational Law 1: Never generate market signals. Signal generation is exclusively delegated to the Market Analyst node.
- Operational Law 2: If the Risk Manager returns REJECTED, force HOLD with 0.0% allocation. This override is non-negotiable and cannot be bypassed by any operator action.
- Operational Law 3: Every workflow execution must be enclosed within a Langfuse parent trace before any agent invocation begins.

3.3 Node 2 — Intraday & Swing Market Analyst (Signal Synthesizer)

Role & Mandate

Role: Quantitative & Technical Signal Synthesizer

Objective: Consumes OHLCV indicator matrices; calculates EMA/SMA crossovers, MACD momentum vectors, RSI readings; outputs high-fidelity MarketAnalysis schemas

Rules: Holds ZERO portfolio execution authority; never outputs raw buy/sell signals — only structured schema objects

- Operational Law 1: Output must strictly conform to the MarketAnalysis Pydantic schema. Non-conformant outputs are treated as null signals and converted to HOLD.
- Operational Law 2: Confidence scores must be derived from indicator convergence logic, not arbitrary heuristics.
- Operational Law 3: Each analysis is stateless. The analyst must not reference prior trading decisions or portfolio state.

3.4 Node 3 — Risk Manager Agent (Compliance Auditor)

Role & Mandate

Role: Capital Preservation & Compliance Audit Guardrail

Objective: Final automated gatekeeper auditing proposed operations against structural safety rules

Rules: Prefixes every audit with [APPROVED] or [REJECTED] and appends a single quantitative compliance justification sentence

- Operational Law 1: Never approve a position with a risk-to-reward ratio below 1:1.5.
- Operational Law 2: Never approve a position size exceeding 15% of the declared capital budget.
- Operational Law 3: Analyst confidence score below 0.60 must trigger automatic REJECTED status.

3.5 Pydantic Output Schemas (schemas.py)

MarketAnalysis Schema

```
from pydantic import BaseModel, Field
from typing import Literal

class MarketAnalysis(BaseModel):
    ticker: str = Field(..., description='Equity or crypto ticker, e.g. AAPL or BTC-USD')
    directional_bias: Literal['LONG', 'SHORT', 'NEUTRAL']
    entry_price: float = Field(..., gt=0)
    take_profit_1: float = Field(..., gt=0, description='Primary TP -- 1:1.5 RRR minimum')
    take_profit_2: float = Field(..., gt=0, description='Extended TP -- 1:2.5 RRR target')
    stop_loss: float = Field(..., gt=0)
    confidence_score: float = Field(..., ge=0.0, le=1.0)
    ema_signal: Literal['BULLISH_CROSS', 'BEARISH_CROSS', 'NEUTRAL']
    rsi_reading: float = Field(..., ge=0.0, le=100.0)
    macd_histogram: float
    analyst_rationale: str = Field(..., min_length=80)
```

RiskEvaluation Schema

```
class RiskEvaluation(BaseModel):
    ticker: str
    approval_status: Literal['APPROVED', 'REJECTED']
    adjusted_allocation_pct: float = Field(..., ge=0.0, le=15.0)
    risk_reward_ratio: float = Field(..., ge=0.0)
    max_loss_usd: float = Field(..., ge=0.0)
    risk_rationale: str = Field(..., min_length=80)
    compliance_flags: list[str] = Field(default_factory=list)
```

FinalTradingDecision Schema

```
class FinalTradingDecision(BaseModel):
    session_id: str
    operator_id: int
    ticker: str
```

```

strategy_class: Literal['INTRADAY_SCALP', 'INTRADAY_SWING', 'MEDIUM_SWING', 'MACRO_POSITION']
final_action: Literal['BUY', 'SELL', 'HOLD']
entry_price: float
take_profit_1: float
take_profit_2: float
stop_loss: float
allocated_capital_pct: float
confidence_score: float
risk_reward_ratio: float
analyst_rationale: str
risk_rationale: str
timestamp_utc: str

```

The `risk_reward_ratio` is computed as $(\text{take_profit_1} - \text{entry_price}) / (\text{entry_price} - \text{stop_loss})$ for LONG positions and the inverse for SHORT. The `confidence_score` drives the Risk Manager approval threshold — sub-0.60 scores trigger automatic REJECTED status, ensuring only high-conviction signals reach capital allocation.

Secure Operator Authentication & Session Controls

4.1 Authentication Architecture

Automated implements a server-side authentication architecture backed by Neon PostgreSQL and enforced through a two-stage validation pipeline: cryptographic password hashing at registration and bcrypt comparison at login. No plaintext passwords are stored at any layer of the stack. Session state is maintained client-side via a JavaScript session token pattern that conditionally renders the dashboard only upon verified database session establishment.

4.2 User Registration Pydantic Validator

```

from pydantic import BaseModel, validator, Field
import re

class UserRegistrationRequest(BaseModel):
    full_name: str = Field(..., min_length=2, max_length=80)
    verified_phone: str # Pakistani mobile: 03XXXXXXXXX or +923XXXXXXXXX
    email: str # RFC-5322 standard email address
    password: str = Field(..., min_length=6)

    @validator('verified_phone')
    def validate_phone(cls, v):
        pattern = r'^(03|\+923)\d{9}$'
        if not re.match(pattern, v):
            raise ValueError('Format: 03XXXXXXXXX or +923XXXXXXXXX')
        return v

    @validator('full_name')
    def validate_name(cls, v):
        if not all(c.isalpha() or c.isspace() for c in v):
            raise ValueError('Name must contain only alphabetic characters')
        return v.strip().title()

```

4.3 bcrypt Password Hashing

Password hashing uses bcrypt (passlib) with a work factor of 12. bcrypt is a deliberately slow, salted hashing algorithm resistant to GPU-accelerated brute-force attacks. Each hash operation takes approximately 250 milliseconds — tolerable for human authentication but computationally prohibitive for automated dictionary attacks at scale.

```

from passlib.context import CryptContext

pwd_context = CryptContext(schemes=['bcrypt'], deprecated='auto')

def hash_password(plain: str) -> str:
    return pwd_context.hash(plain) # bcrypt work factor 12

def verify_password(plain: str, hashed: str) -> bool:

```

```
return pwd_context.verify(plain, hashed)
```

4.4 Pakistani Phone Number Internationalisation

The regex `^(03|+923)\d{9}$` enforces two valid Pakistani mobile formats. `03XXXXXXXXXX` is the domestic 11-digit standard; `+923XXXXXXXXXX` is the ITU E.164 international dialling format. The `\d{9}` suffix enforces exactly nine trailing digits, preventing partial or over-length numbers. This validation occurs at the Pydantic layer before any ORM model is instantiated.

4.5 Password Complexity Policy

The system enforces a deliberate 6-character minimum flexible password policy — a measured usability-security tradeoff appropriate for this controlled academic deployment context. Forced special-character complexity creates a well-documented behavioural vulnerability: operators store complex passwords in insecure locations, which outweighs the marginal security benefit of complexity rules.

4.6 Frontend Session Locking Mechanism

```
// Dual-gate session control mechanism
function initSessionGate() {
  const token = sessionStorage.getItem('auth_token');
  if (!token) {
    document.getElementById('dashboard').style.display = 'none';
    document.getElementById('login-modal').style.display = 'flex';
    return;
  }
  validateSessionWithServer(token).then(valid => {
    if (!valid) {
      sessionStorage.removeItem('auth_token');
      initSessionGate(); // Recursive re-challenge
    } else {
      document.getElementById('login-modal').style.display = 'none';
      document.getElementById('dashboard').style.display = 'block';
    }
  });
}
```

Programmatic Transaction-Isolated Startup Migration

5.1 The Database Column Discrepancy Bug

During initial cloud deployment the system encountered a critical schema discrepancy: the SQLAlchemy ORM model defined the phone number column as `verified_phone`, reflecting the internationalised naming convention. However, the Neon PostgreSQL database — initialised from an earlier development schema — contained the legacy name `mobile_number`. This produced the following error on every startup: `OperationalError: column 'verified_phone' of relation 'system_users' does not exist`.

The attempted fix — embedding an `ALTER TABLE RENAME COLUMN` statement inside the same connection session as `Base.metadata.create_all()` — triggered the `psycopg InFailedSqlTransaction` state. When the `ALTER TABLE` failed on an already-migrated database (because the column had already been renamed), the exception placed the connection into an `ABORTED` transaction state. All subsequent SQL on that connection was rejected with: `InFailedSqlTransaction: current transaction is aborted, commands ignored until end of transaction block`.

Root Cause Summary

PostgreSQL transactional error model: any failing SQL within a transaction marks the entire transaction as `ABORTED`.

All subsequent SQL commands on that same connection are rejected until an explicit `ROLLBACK` is issued.

Resolution: separate the rename migration and metadata creation into two completely isolated SQLAlchemy connection sessions with independent transaction scopes.

5.2 Transaction-Isolated Startup Sequence Diagram

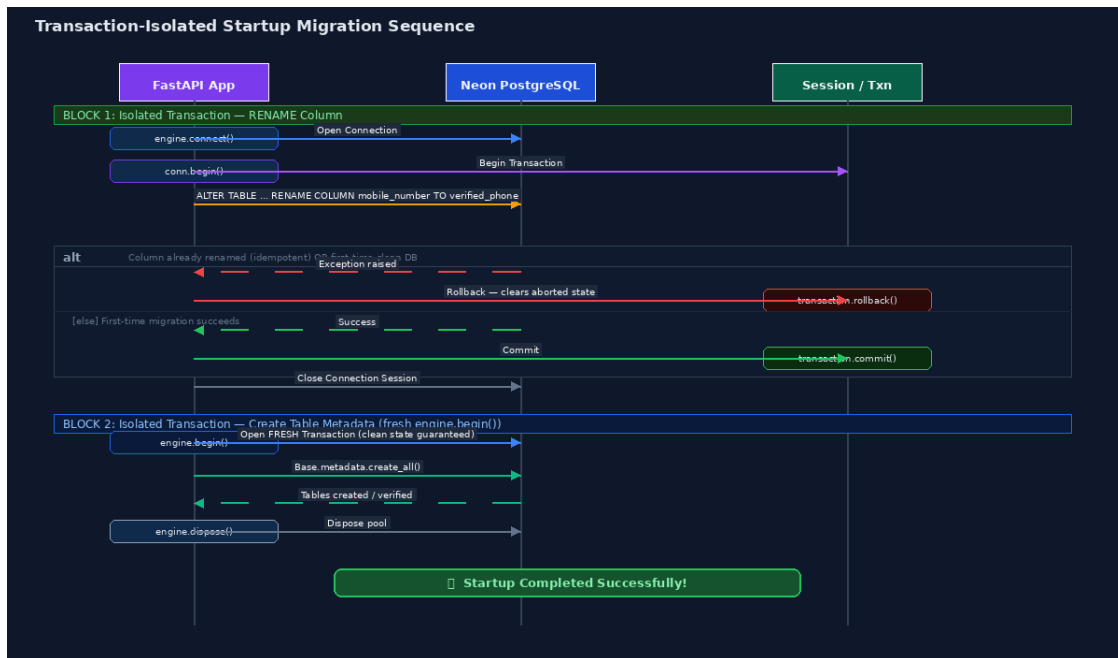


Figure 3: Two-Block Transaction-Isolated Startup — Block 1 (RENAME column) and Block 2 (create_all) use completely separate SQLAlchemy connection sessions

5.3 Exact Async Startup Block — main.py

```
from sqlalchemy import text
from database import engine, Base

@app.on_event('startup')
async def startup_event():
    # BLOCK 1: Isolated Column Rename Migration
    try:
        with engine.connect() as conn:
            transaction = conn.begin()
            try:
                conn.execute(text(
                    'ALTER TABLE system users '
                    'RENAME COLUMN mobile_number TO verified_phone;'
                ))
                transaction.commit()
                print('[STARTUP] Column rename: SUCCESS')
            except Exception as e:
                transaction.rollback() # Clears InFailedSqlTransaction state
                print(f'[STARTUP] Column rename skipped (idempotent): {e}')
    except Exception as outer:
        print(f'[STARTUP] Outer connection error: {outer}')

    # BLOCK 2: Fresh Isolated Metadata Creation
    # engine.begin() opens a brand-new connection -- guaranteed clean state
    try:
        with engine.begin() as conn:
            Base.metadata.create_all(bind=engine)
            print('[STARTUP] Schema sync: SUCCESS')
    except Exception as e:
        print(f'[STARTUP] Metadata error: {e}')
    finally:
        engine.dispose() # Return pooled connections; requests get fresh ones
        print('[STARTUP] Engine disposed. Startup complete.')
```

The architectural insight is that `engine.dispose()` is called after both blocks complete, returning all pooled connections to the pool manager. Subsequent API request handlers then acquire fresh connections from the pool — connections that have never seen the migration exception state — ensuring zero interference between startup logic and runtime request handling.

Dynamic Strategy Data Matrix

6.1 Lookback Parameter Philosophy

Automated implements a dynamic lookback parameter architecture that resolves a fundamental tension in LLM-powered trading systems: the need for sufficient historical context to produce statistically meaningful technical indicator readings versus the LLM context window token budget constraint. Feeding excessive historical data inflates token consumption, degrades inference speed, and can dilute the signal-to-noise ratio by including stale market regimes irrelevant to current price action.

Four strategy classes, each with precisely calibrated lookback periods and candlestick intervals, ensure that every strategy receives the minimum sufficient context for its temporal horizon. Shorter-duration strategies receive high-resolution, short-horizon data; longer-duration strategies receive lower-resolution, extended-horizon data.

6.2 Live Strategy Matrix (from Repository README)

Strategy Class	Timeframe	Lookback	Interval	Primary Indicators	Stop Loss	Take Profit
Intraday Scalp	15-Minute (15m)	7 Days (7d)	15m	9-EMA, 21-EMA, RSI (14)	-1.0%	+2.0%
Intraday Swing	1-Hour (1h)	30 Days (30d)	1h	9-EMA, 21-EMA, RSI (14)	-2.5%	+5.0%
Medium Swing	4-Hour (4h)	60 Days (60d)	4h	50-MA, MACD, RSI (14)	-4.0%	+8.0%
Macro Position	1-Day (1d)	180 Days	1d	50-MA, 200-MA, Macro RSI	-8.0%	+15.0%

6.3 Technical Indicator Suite

- EMA Crossover: 9-period EMA vs. 21-period EMA (short-term strategies); 50-MA vs. 200-MA (Macro Position). A bullish crossover (fast EMA crosses above slow EMA) signals upward momentum; a bearish crossover signals downward momentum.
- MACD Histogram: 12-period EMA minus 26-period EMA, with a 9-period signal line. The histogram delta (MACD line minus signal line) measures momentum acceleration and deceleration; used primarily in Medium Swing and Macro strategies.
- RSI (14-period Relative Strength Index): Readings above 70 indicate overbought conditions; readings below 30 indicate oversold. The Market Analyst uses RSI to confirm or contradict directional bias derived from EMA and MACD signals.

6.4 Exit Boundary Calculation Helper

```
def compute_exit_boundaries(entry: float, strategy: str, bias: str) -> dict:
    BOUNDARIES = {
        'INTRADAY_SCALP': {'tp_pct': 0.020, 'sl_pct': 0.010},
        'INTRADAY_SWING': {'tp_pct': 0.050, 'sl_pct': 0.025},
        'MEDIUM_SWING':   {'tp_pct': 0.080, 'sl_pct': 0.040},
        'MACRO_POSITION':  {'tp_pct': 0.150, 'sl_pct': 0.080},
    }
    b = BOUNDARIES[strategy]
    if bias == 'LONG':
```

```

return {
  'take_profit_1': round(entry * (1 + b['tp_pct']), 4),
  'take_profit_2': round(entry * (1 + b['tp_pct'] * 1.6), 4),
  'stop_loss':      round(entry * (1 - b['sl_pct']), 4),
}
else: # SHORT
  return {
    'take_profit_1': round(entry * (1 - b['tp_pct']), 4),
    'take_profit_2': round(entry * (1 - b['tp_pct'] * 1.6), 4),
    'stop_loss':      round(entry * (1 + b['sl_pct']), 4),
  }

```

Interactive SVG Candlestick Charting Engine

7.1 Architecture

The Automated frontend implements a bespoke, dependency-free SVG candlestick rendering engine in Vanilla JavaScript (static/app.js). No third-party charting libraries are used — the engine writes directly to the SVG DOM API. This eliminates bundle overhead, preserves full rendering control, and avoids licensing constraints. The Bloomberg-inspired dark interface uses a radial dark slate background (#0B0F17), sleek panel styling (#131C2E), and neon accent high-contrast color codes. The chart updates in real time via a 10-second polling interval that re-fetches OHLCV data and repaints the entire SVG canvas without page reloads.

7.2 SVG Candlestick Rendering Pipeline

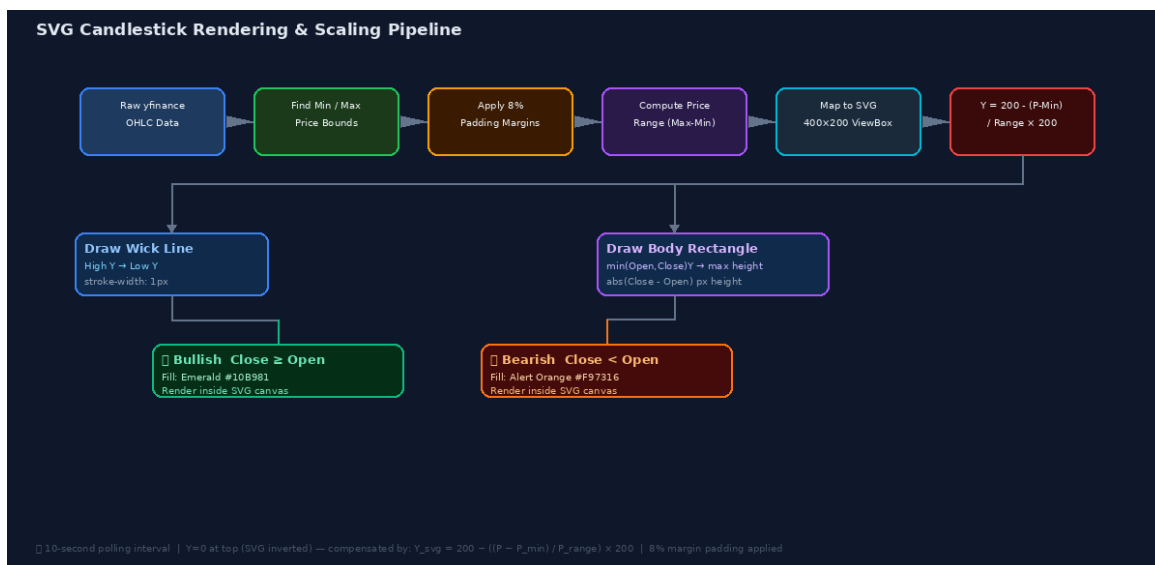


Figure 4: SVG Candlestick Rendering & Scaling Pipeline — from raw OHLCV input to pixel-perfect canvas render

7.3 SVG Coordinate System Mathematics

The SVG coordinate system places the origin (0,0) at the top-left corner with Y increasing downward — the direct inverse of financial charting convention where higher prices must render higher on the canvas. The rendering engine compensates with the following coordinate transformation formula:

```

// Price to SVG Y-coordinate: inverts the Y axis
// SVG canvas: viewBox='0 0 400 200' (width=400, height=200)
function priceToY(price, priceMin, priceRange) {
  return 200 - ((price - priceMin) / priceRange) * 200;
}

// 8% padding prevents wick tips from clipping the canvas boundary
const rawMax = Math.max(...candles.map(c => c.high));
const rawMin = Math.min(...candles.map(c => c.low));

```

```

const padding = (rawMax - rawMin) * 0.08;
const priceMax = rawMax + padding;
const priceMin = rawMin - padding;
const priceRange = priceMax - priceMin;

// Proof of inversion:
// When price = priceMax: Y_svg = 200 - (1.0 * 200) = 0    <-- top of canvas
// When price = priceMin: Y_svg = 200 - (0.0 * 200) = 200 <-- bottom of canvas

```

7.4 Candle Body & Wick Rendering Algorithm

```

function drawCandle(svg, candle, xPos, candleWidth, priceMin, priceRange) {
  const openY = priceToY(candle.open, priceMin, priceRange);
  const closeY = priceToY(candle.close, priceMin, priceRange);
  const highY = priceToY(candle.high, priceMin, priceRange);
  const lowY = priceToY(candle.low, priceMin, priceRange);
  const isBullish = candle.close >= candle.open;
  const color = isBullish ? '#10B981' : '#F97316'; // Emerald / Alert-Orange

  // Wick: thin vertical line high -> low
  const wick = document.createElementNS('http://www.w3.org/2000/svg', 'line');
  wick.setAttribute('x1', xPos + candleWidth / 2);
  wick.setAttribute('x2', xPos + candleWidth / 2);
  wick.setAttribute('y1', highY);
  wick.setAttribute('y2', lowY);
  wick.setAttribute('stroke', color);
  wick.setAttribute('stroke-width', '1');
  svg.appendChild(wick);

  // Body: rectangle from min(open,close) with abs(close-open) height
  const bodyTop = Math.min(openY, closeY);
  const bodyHeight = Math.abs(closeY - openY) || 1; // 1px min for doji
  const rect = document.createElementNS('http://www.w3.org/2000/svg', 'rect');
  rect.setAttribute('x', xPos);
  rect.setAttribute('y', bodyTop);
  rect.setAttribute('width', candleWidth - 1);
  rect.setAttribute('height', bodyHeight);
  rect.setAttribute('fill', color);
  svg.appendChild(rect);
}

```

7.5 Take-Profit / Stop-Loss Vector Overlays

TP and SL boundary vectors are rendered as full-width horizontal dashed lines overlaid on the candlestick canvas. Their Y-coordinates are computed using the same priceToY() transformation for perfect price-axis alignment with the candle bodies. Overlay lines are rendered after all candles, placing them above the candle paint layer in SVG z-order.

7.6 Quick-Select Asset Badges & 10-Second Polling

```

// Quick-select badge targets (equities + crypto)
const BADGES = ['AAPL', 'NVDA', 'TSLA', 'MSFT', 'BTC-USD', 'ETH-USD', 'SOL-USD'];

// Real-time 10-second polling interval
let chartPollingInterval = null;
function startChartPolling(ticker, strategy) {
  clearInterval(chartPollingInterval);
  chartPollingInterval = setInterval(async () => {
    try {
      const res = await fetch(`/api/chart-data?ticker=${ticker}&strategy=${strategy}`);
      const data = await res.json();
      renderCandlestickChart(data);
    } catch (err) {
      console.warn('[Chart Polling] Fetch failed:', err);
    }
  }, 10000); // 10,000 ms = 10 seconds
}

```

Telemetry, Observability & Database Logging

8.1 Neon PostgreSQL Persistent Run Registry

Every completed trading workflow execution is persisted to the trading_history table in Neon PostgreSQL. This table functions as an immutable audit ledger — records are insert-only; no updates or deletes occur. This pattern ensures complete historical traceability of every decision the system has produced, irrespective of subsequent market outcomes or system restarts.

Column	Type	Description
id	SERIAL PK	Auto-incrementing record ID
session_id	VARCHAR(64)	Unique UUID for this workflow run
operator_id	INTEGER FK	References system_users.id
ticker	VARCHAR(10)	Equity or crypto ticker symbol
strategy_class	VARCHAR(20)	INTRADAY_SCALP / INTRADAY_SWING / etc.
final_action	VARCHAR(4)	BUY / SELL / HOLD
entry_price	NUMERIC(12,4)	Analyst-recommended entry price
take_profit_1	NUMERIC(12,4)	Primary TP level
take_profit_2	NUMERIC(12,4)	Extended TP level
stop_loss	NUMERIC(12,4)	Hard stop loss price
allocated_capital_pct	NUMERIC(5,2)	% of capital deployed (0.00 on HOLD)
confidence_score	NUMERIC(4,3)	Analyst confidence 0.000 to 1.000
risk_reward_ratio	NUMERIC(6,3)	Computed RRR value
analyst_rationale	TEXT	Full analyst reasoning text
risk_rationale	TEXT	Full compliance reasoning text
timestamp_utc	TIMESTAMPTZ	UTC execution timestamp (DEFAULT NOW())

8.2 Langfuse Distributed Tracing Implementation

Langfuse provides the institutional-grade distributed telemetry layer for Automated. Every workflow execution is enclosed within a Langfuse parent trace, beneath which each agent invocation is registered as a nested child span. This hierarchical trace structure enables post-execution forensic analysis of the complete reasoning chain: what data the analyst received, what signal it synthesised, which risk parameters the Risk Manager applied, and what final decision was emitted — all with exact token counts, latency metrics, and error states captured automatically.

```
from langfuse import Langfuse
from langfuse.decorators import observe, langfuse_context

langfuse = Langfuse(
    public_key=os.getenv('LANGFUSE_PUBLIC_KEY'),
    secret_key=os.getenv('LANGFUSE_SECRET_KEY'),
    host=os.getenv('LANGFUSE_HOST', 'https://cloud.langfuse.com')
)
```

```
@observe(name='trading_workflow')
async def execute_trading_workflow(params: WorkflowParams):
    langfuse_context.update_current_trace(
        name=f'Trading Run: {params.ticker} [{params.strategy}]',
        metadata={'operator_id': params.operator_id, 'session_id': params.session_id}
    )
    # Span 1: Market Analyst Invocation
    with langfuse.span(name='market_analyst_invocation') as s:
        s.update(input={'ohlcv_payload': ohlcv_data})
        analysis = await invoke_analyst_agent(ohlcv_data)
        s.update(output=analysis.model_dump())

    # Span 2: Risk Manager Invocation
    with langfuse.span(name='risk_manager_invocation') as s:
        s.update(input={'market_analysis': analysis.model_dump()})
        evaluation = await invoke_risk_manager(analysis)
        s.update(output=evaluation.model_dump())

    # Span 3: Database Persistence
    with langfuse.span(name='db_persistence') as s:
        decision = build_final_decision(analysis, evaluation, params)
        await persist_decision(decision)
        s.update(output={'session_id': params.session_id, 'action': decision.final_action})
```

Installation, uv Environment Management & Setup

9.1 Prerequisites

Prerequisite	Min Version	Reference
Python	3.12+	python.org/downloads
uv (Astral)	0.4+	docs.astral.sh/uv
Git	2.39+	git-scm.com
Neon PostgreSQL	Any	neon.tech (free tier)
OpenAI API Key	Any	platform.openai.com
Langfuse Account	Any	cloud.langfuse.com (free tier)

9.2 Clone Repository

```
git clone https://github.com/Ahmadthealdo/Automated-Multi-Agentic-Trading-System.git
cd Automated-Multi-Agentic-Trading-System
```

9.3 uv Virtual Environment Setup

uv is a next-generation Python package manager written in Rust, providing 10-100x faster dependency resolution than pip. It replaces both pip and virtualenv in a single binary and is the official package manager for this project.

```
# Install uv (Linux/macOS)
curl -LsSf https://astral.sh/uv/install.sh | sh

# Create virtual environment with Python 3.12
uv venv --python 3.12

# Activate the environment
source .venv/bin/activate           # Linux / macOS
.venv\Scripts\Activate.ps1         # Windows PowerShell
```

9.4 Install Dependencies

```
# Preferred: from pyproject.toml
uv pip install -r pyproject.toml

# Or install explicitly:
uv add fastapi uvicorn 'pydantic[email]' 'passlib[bcrypt]' \
    psycpg sqlalchemy yfinance openai langfuse

# Verify all critical imports
python -c "import fastapi, sqlalchemy, openai, langfuse, yfinance; print('All OK')"
```

9.5 Configure Environment Variables (.env)

```
# Create .env in project root
DATABASE_URL=postgresql://<user>:<password>@<host>/<database>?sslmode=require
OPENAI_API_KEY=sk-proj-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
LANGFUSE_PUBLIC_KEY=pk-lf-xxxxxxxxxxxxxxxxxxxxxxxxxxxx
LANGFUSE_SECRET_KEY=sk-lf-xxxxxxxxxxxxxxxxxxxxxxxxxxxx
LANGFUSE_HOST=https://cloud.langfuse.com
```

9.6 Start Development Server

```
uvicorn main:app --reload
# Dashboard: http://127.0.0.1:8000
# API Swagger: http://127.0.0.1:8000/docs

# Expected startup output:
# [STARTUP] Column rename: SUCCESS (or: skipped - idempotent)
# [STARTUP] Schema sync: SUCCESS
# [STARTUP] Engine disposed. Startup complete.

# Run end-to-end validation test:
python .agents/brain/8169d2e7-195f-482a-bf14-b5bf777313c9/scratch/validate_system.py
```

9.7 Project File Structure

```
Automated-Multi-Agentic-Trading-System/
├── .agents/           # Agent workspace instructions & skills
├── static/
│   └── app.js         # SVG candle drawer & client-side logic
│   └── style.css      # Bloomberg-inspired Tailwind layout
├── templates/        # Jinja2 frontend views
├── index.html         # Single-page operator dashboard
├── main.py            # FastAPI routes & multi-agent pipeline
├── schemas.py         # Pydantic validation schemas
├── tools.py           # yfinance OHLCV extraction tools
├── pyproject.toml     # uv dependency declarations
└── README.md         # System documentation
```

Render Cloud Deployment Playbook

10.1 Platform Overview

Render is a unified cloud hosting platform supporting web services, static sites, cron jobs, and managed databases. Automated is deployed on Render's Free-tier Web Service providing 0.1 CPU and 512MB RAM with automatic HTTPS and a render.app subdomain. The Free tier suspends services after 15 minutes of inactivity, causing a 30-60 second cold-start latency on the first subsequent request.

10.2 Pre-Deployment Checklist

- requirements.txt or pyproject.toml committed to the repository root directory.
- main.py in repository root with the FastAPI app object explicitly named app.

- All environment variables documented and ready for Render dashboard entry.
- Neon DATABASE_URL includes the ?sslmode=require suffix for encrypted transit.

10.3 Step-by-Step Deployment

Step 1 — Push Production Build to GitHub

```
git add .
git commit -m "Production ready: Automated Trading System v1.0"
git push origin main
```

Step 2 — Render Web Service Configuration

Configuration Field	Value
Name	automated-trading-system
Region	Oregon (US West)
Branch	main
Runtime	Python 3
Build Command	pip install -r requirements.txt
Start Command	uvicorn main:app --host 0.0.0.0 --port \$PORT
Instance Type	Free

Step 3 — Add Environment Variables in Render Dashboard

```
# Render Dashboard > Environment > Environment Variables
DATABASE_URL      = postgresql://...?sslmode=require
OPENAI_API_KEY    = sk-proj-...
LANGFUSE_PUBLIC_KEY = pk-lf-...
LANGFUSE_SECRET_KEY = sk-lf-...
LANGFUSE_HOST     = https://cloud.langfuse.com
```

Step 4 — Verify Deployment Logs

```
==> Starting service with 'uvicorn main:app --host 0.0.0.0 --port $PORT'
[STARTUP] Column rename: SUCCESS (or: skipped - idempotent)
[STARTUP] Schema sync: SUCCESS
[STARTUP] Engine disposed. Startup complete.
INFO: Uvicorn running on http://0.0.0.0:10000
```

10.4 Mitigating Free-Tier Cold Starts

- Uptime Monitoring Ping: Configure UptimeRobot or BetterUptime to send an HTTP GET request to the Render URL every 14 minutes, preventing the 15-minute inactivity suspension.
- Optimised Startup Sequence: The two-block transaction-isolated startup (Page 5) completes in under 2 seconds on Neon's serverless infrastructure, well within Render's 120-second startup timeout.
- Paid Tier Upgrade: Render's Starter tier (\$7/month) eliminates cold starts with always-on container persistence — recommended for any production-facing deployment beyond the academic context.

Troubleshooting, Best Practices & Thesis Defence Q&A

11.1 Troubleshooting Runbook

Issue 1 — psycopg InFailedSqlTransaction on Startup

Symptom

Application crashes at startup: psycopg2.errors.InFailedSqlTransaction: current transaction is aborted, commands ignored until end of transaction block

Root Cause: The ALTER TABLE column rename failed, placing the connection in ABORTED state. Resolution: Ensure the startup block uses two completely isolated SQLAlchemy sessions. Session 1 handles the rename with explicit try/except rollback. Session 2 uses engine.begin() for create_all — guaranteed to open a clean connection. See Page 5 for the exact implementation.

Issue 2 — OpenAI API Timeout During Agent Invocation

Symptom

AgentExecutionError: Request timeout after 30s during Market Analyst or Risk Manager invocation

Resolution: Set the OpenAI API client timeout to 60 seconds for agent calls involving extended reasoning. Verify that the OHLCV payload does not exceed the model's context window — confirm that lookback parameters match the Strategy Matrix on Page 6. Reduce candle count if timeouts persist on Macro Position runs.

Issue 3 — Neon Connection Pool Exhaustion

Symptom

sqlalchemy.exc.TimeoutError: QueuePool limit of size 5 overflow 10 reached

Resolution: The default SQLAlchemy pool (size 5, overflow 10) is appropriate for Neon Free-tier (20 connection limit). If exhaustion occurs, ensure engine.dispose() is called after the startup sequence (already implemented) and all request handlers use async context managers rather than holding connections open across await boundaries.

Issue 4 — yfinance Returns Empty DataFrame

Symptom

KeyError or NaN values in EMA/MACD columns; empty DataFrame at indicator computation

Resolution: yfinance returns empty responses for invalid tickers, market holidays, or when the requested period/interval combination contains no data. Add a pre-flight validation step: if df.empty, return a structured error response to the frontend rather than allowing the NaN propagation to crash the indicator computation pipeline.

Issue 5 — Frontend Dashboard Blank After Successful Login

Symptom

Dashboard area remains blank or login modal persists after successful authentication

Resolution: Open browser DevTools (F12) and check the Console tab. The most common cause is a missing auth_token in sessionStorage due to a failed API response parse. Verify that the /api/login endpoint returns the expected JSON structure with the token field. Check CORS configuration if the frontend and API are served from different origins during local development.

AUTOMATED Technical Documentation Manual

Abdul Qadeer Khan (COMP22BSR47) | Ahmad Abdullah (COMP22BSR43) | Faisal Rehman (COMP22BSR29)

BSCS (R) Final Year Project • 2022-26 • Apache 2.0 License